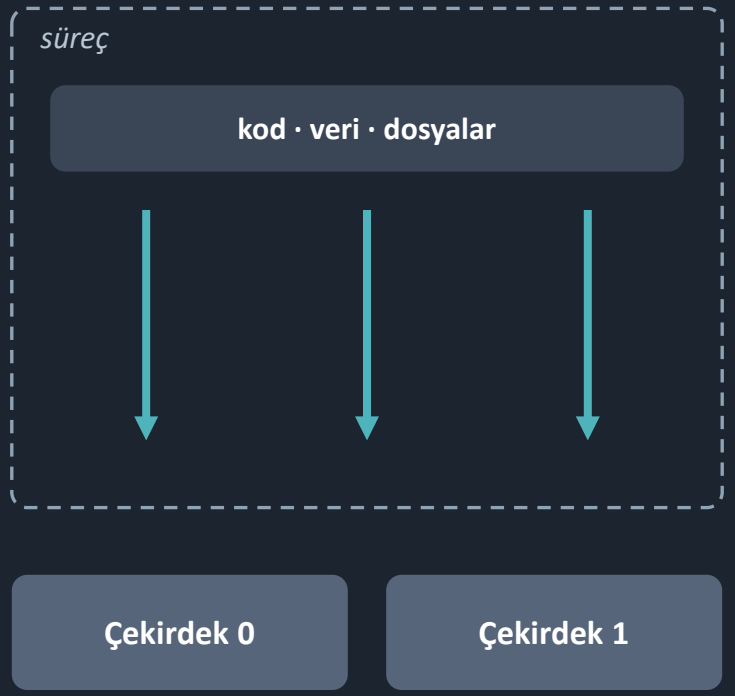
KARADENİZ TEKNİK ÜNİVERSİTESİ

Yazılım Geliştirme Bölümü

İŞLETİM SİSTEMLERİ

Hafta 4

İş Parçacıkları ve Çok Çekirdekli Programlama



Dr. Öğr. Üyesi Sayyad Alizadeh

Bu Haftanın Yol Haritası

1

İş parçacığı kavramı

Bir süreç içinde birden çok yürütme akışı olabilir mi?

2

Neden çok iş parçacığı?

Yanıt verebilirlik, paylaşım, ekonomi, ölçeklenebilirlik

3

Eşzamanlılık ve paralellik

Aynı anda ilerlemek ile aynı anda çalışmak aynı mı?

4

Amdahl Yasası

Çekirdek sayısını artırmak programı ne kadar hızlandırır?

5

Kullanıcı ve çekirdek iş parçacıkları

Çoktan-bire, birebir, çoktan-çoka modeller

6

İş parçacığı kütüphaneleri

Pthreads, Windows ve Java

7

Örtük iş parçacığı

İş parçacığı havuzu ve OpenMP

8

Sorunlar ve gerçek sistemler

fork/exec, sinyal, iptal, TLS · Windows, Linux

Öğrenme Çıktıları

Bu haftanın sonunda öğrenciler:

1. İş parçacığını tanımlar; paylaşılan ve özel kaynakları ayırt eder
2. Süreç ile iş parçacığını maliyet, paylaşım ve yalıtım açısından karşılaştırır
3. Eşzamanlılık ile paralelliği; veri ile görev paralelliğini ayırt eder
4. Amdahl Yasası ile teorik hızlanmayı hesaplar ve yorumlar
5. Kullanıcı/çekirdek iş parçacıklarını ve üç eşleme modelini karşılaştırır
6. Pthreads ile iş parçacığı oluşturup bekleyen kısa programları açıklar

Geçen Haftadan Bu Haftaya

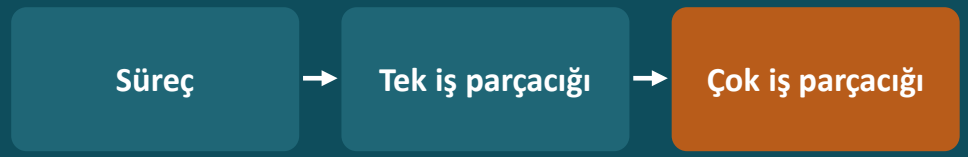
HAFTA 3'TE GÖRDÜKLERİMİZ

Süreç ve PCB	**Bağlam değiştirme (ek yük)**
fork() ve exec()	**IPC: paylaşımlı bellek, mesaj**

- Şimdiye kadar her sürecin tek bir program sayacı vardı
- Yeni süreç oluşturmak ve süreçler arasında geçiş yapmak maliyetlidir
- Süreçler arası veri paylaşımı IPC gerektirir

BU HAFTANIN TEMEL SORUSU

Bir süreç içinde yalnızca bir yürütme akışı mı olabilir?



Süreç kavramı yeniden anlatılmaz; bu hafta bir sürecin içindeki yürütme akışlarına odaklanılır.

İş Parçacığı (Thread) Nedir?

İş parçacığı: CPU kullanımının temel birimidir. Bir süreç içinde bağımsız olarak zamanlanabilen bir yürütme akışıdır.

Her iş parçacığının kendine ait olanlar

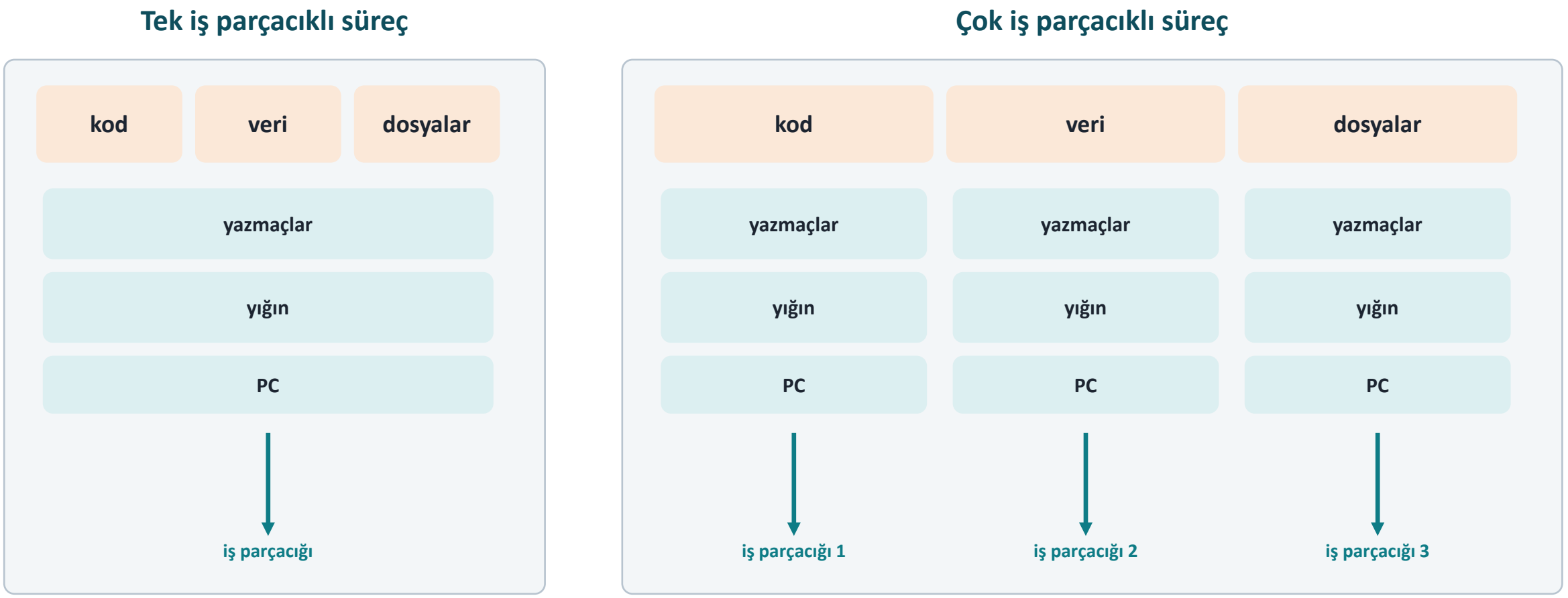
İş parçacığı kimliği	Program sayacı	Yazmaç kümesi	Yığın
thread ID	sıradaki komut	CPU yazmaç değerleri	işlev çağrıları, yerel değişkenler

Aynı süreçteki diğer iş parçacıklarıyla paylaştıkları

Kod (metin) bölümü	Veri bölümü ve öbek	Açık dosyalar, sinyaller ve diğer İS kaynakları

Geleneksel (ağır) süreç = tek iş parçacıklı süreç. Kaynaklar: Silberschatz vd., Bölüm 4.1; Tanenbaum & Bos, Bölüm 2.2.2

Tek İş Parçacıklı ve Çok İş Parçacıklı Süreç



Turuncu: süreç düzeyinde paylaşılan · Camgöbeği: her iş parçacığına özel

Silberschatz vd., Şekil 4.1

Hangi Kaynak Paylaşılır, Hangisi Özeldir?

Süreç düzeyinde (tüm iş parçacıkları paylaşır)	İş parçacığı düzeyinde (her birine özel)
Adres uzayı: kod ve veri bölümleri	İş parçacığı kimliği
Öbek (heap): dinamik ayrılan bellek	Program sayacı
Global değişkenler	Yazmaç kümesi
Açık dosyalar	Yığın (yerel değişkenler, çağrı zinciri)
Çocuk süreçler, sinyaller ve sinyal işleyicileri	Durum (çalışıyor, hazır, engellenmiş…)

Sonuç: Bir iş parçacığının yazdığı global veri diğerleri tarafından hemen görülür.

Neden ayrı yığın? Her iş parçacığı farklı işlevleri çağırır; kendi çağrı geçmişine ihtiyaç duyar.

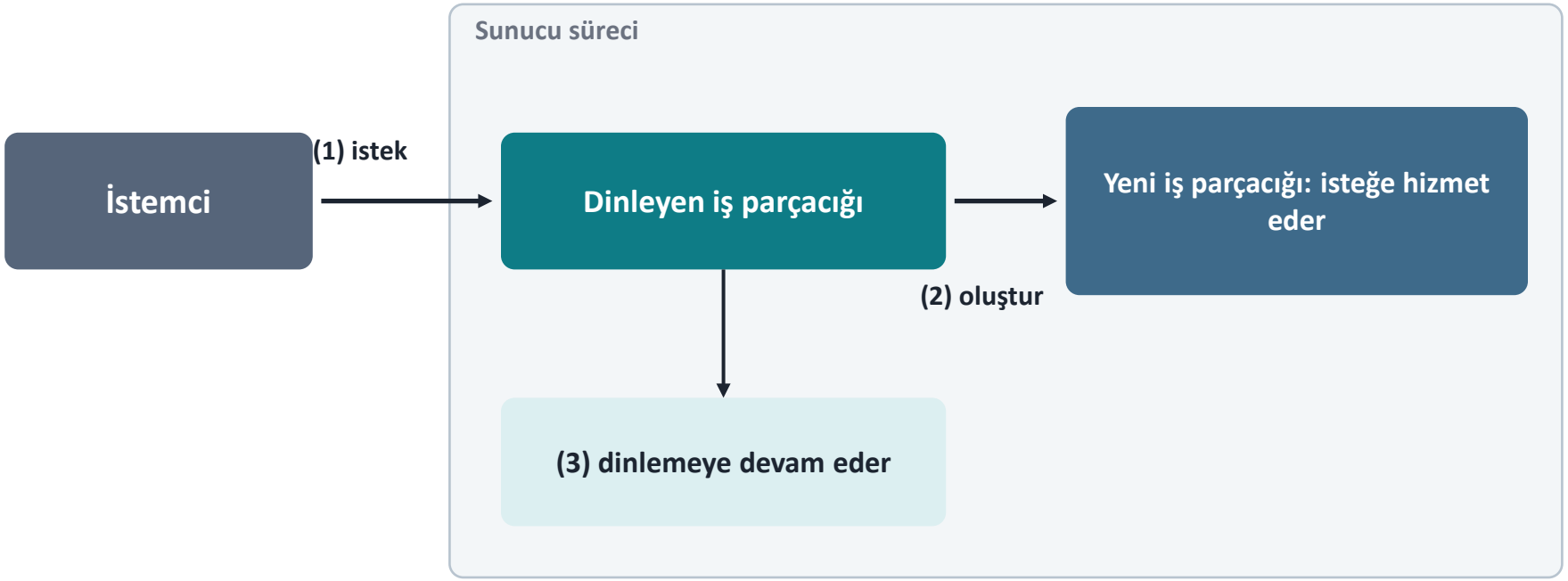
Tanenbaum & Bos, Şekil 2-12 temel alınmıştır

Süreç ile İş Parçacığının Karşılaştırması

Ölçüt	Süreç	İş parçacığı (aynı süreç içinde)
Adres uzayı	Her sürecin kendi adres uzayı vardır	Sürecin adres uzayını paylaşır
Kaynak sahipliği	Kaynakların sahibidir (kaynak kabı)	Sürecin kaynaklarını kullanır
Oluşturma maliyeti	Görece yüksek: yeni adres uzayı ve kaynaklar	Genellikle daha düşük olabilir
Bağlam değiştirme	Adres uzayı da değişir	Genellikle daha düşük maliyetli olabilir
İletişim	IPC gerekir (paylaşımlı bellek, mesaj…)	Ortak bellek üzerinden doğrudan
Yalıtım	Güçlü: bir süreç diğerinin belleğine erişemez	Zayıf: bir iş parçacığının hatası tüm süreci etkileyebilir

Ödünleşim: Paylaşım verimlilik getirir ama yalıtımı azaltır. Ortak veriye eşzamanlı erişimin doğurduğu sorunlar Hafta 6'nın konusudur.

Motivasyon: Çok İş Parçacıklı Bir Web Sunucusu



Alternatif?

Her istek için yeni süreç: işe yarar ama süreç oluşturmak zaman ve kaynak açısından maliyetlidir.

Aynı işi yapan yeni süreç yerine yeni iş parçacığı: sunucu kodunu, verisini ve dosyalarını paylaşır; oluşturma daha ekonomik olabilir.

Silberschatz vd., Bölüm 4.1.1 ve Şekil 4.2 temel alınmıştır

Çok İş Parçacıklı Programlamanın Dört Faydası

1

Yanıt verebilirlik

Responsiveness

Uzun bir iş sürerken uygulama kullanıcıya yanıt vermeyi sürdürür

Bir iş parçacığı arayüzü yönetirken diğeri ağdan veri indirir

2

Kaynak paylaşımı

Resource Sharing

İş parçacıkları sürecin belleğini ve kaynaklarını varsayılan olarak paylaşır

IPC kurmadan ortak veri yapısına erişim

3

Ekonomi

Economy

İş parçacığı oluşturmak ve aralarında geçiş yapmak genellikle süreçten daha ekonomiktir

Kitapta: Solaris'te süreç oluşturma yaklaşık 30 kat daha yavaş

4

Ölçeklenebilirlik

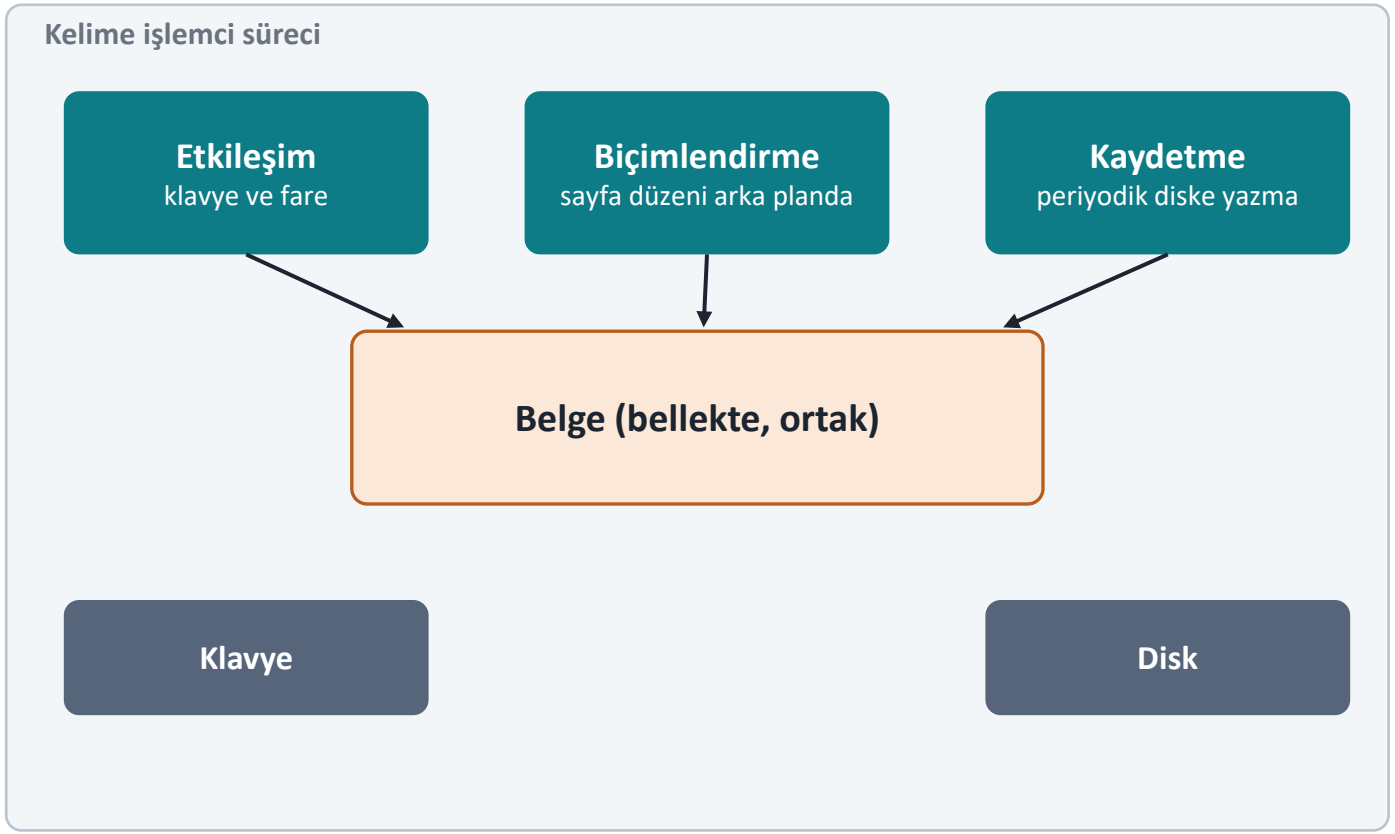
Scalability

İş parçacıkları farklı çekirdeklerde paralel çalışabilir

Tek iş parçacıklı süreç, kaç çekirdek olursa olsun tek çekirdekte çalışır

Kavramsal örnek: tarayıcıda bir iş parçacığı arayüzü, biri ağ verisini, biri sayfa çizimini yürütebilir. (Belirli bir tarayıcının güncel mimarisini temsil etmez.)

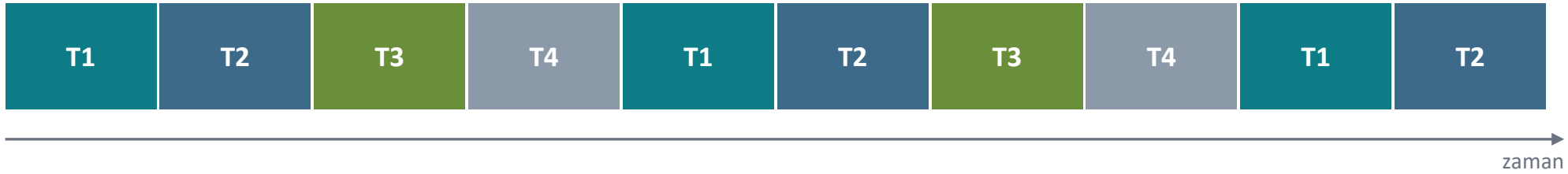
Kavramsal Örnek: Üç İş Parçacıklı Kelime İşlemci



- Üç iş tek bir belge üzerinde çalışır: ortak bellek gerekir
- Üç ayrı süreç olsaydı belgeyi paylaşmak zorlaşırdı
- Yedekleme diske yazarken kullanıcı yazmayı sürdürebilir

Tanenbaum & Bos, Bölüm 2.2.1 (Şekil 2-7) temel alınmıştır

Eşzamanlılık (Concurrency): Tek Çekirdekte



Eşzamanlılık: Birden çok görevin aynı zaman aralığında ilerlemesine izin verilmesi. Tek çekirdekte iş parçacıkları zaman içinde iç içe geçerek yürütülür.

Dikkat

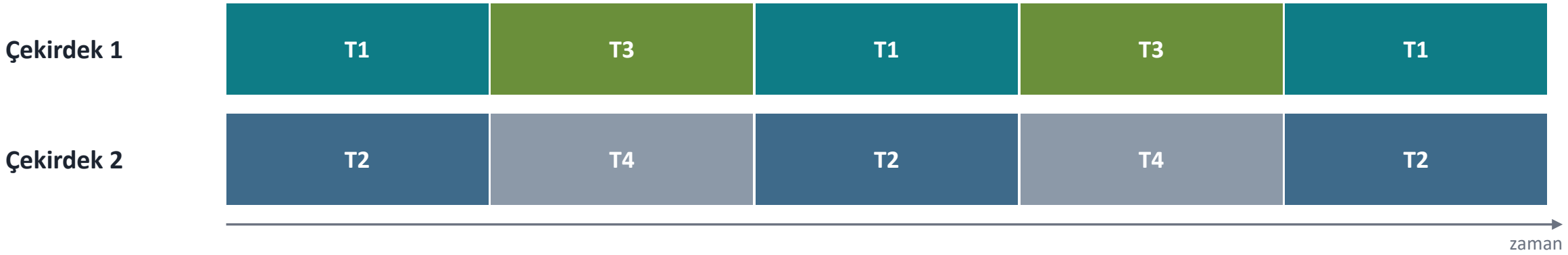
Herhangi bir anda yalnızca bir iş parçacığı fiziksel olarak çalışır. Hızlı geçişler nedeniyle aynı anda çalışıyormuş gibi görünürler.

Hafta 3 bağlantısı

Bu iç içe geçme, Hafta 3'teki bağlam değiştirmeye dayanır; sıranın nasıl seçildiği Hafta 5'in konusudur.

Silberschatz vd., Bölüm 4.2 ve Şekil 4.3 temel alınmıştır

Paralellik (Parallelism): Çok Çekirdekte



Paralellik: Birden çok görevin gerçekten aynı anda yürütülmesi. Her çekirdeğe ayrı bir iş parçacığı atanabilir.

Ölçüt	Eşzamanlılık	Paralellik
Ne demek?	Aynı zaman aralığında ilerleme	Aynı anda fiziksel yürütme
Tek çekirdekte mümkün mü?	Evet	Hayır

Paralellik olmadan eşzamanlılık mümkündür. Silberschatz vd., Bölüm 4.2 ve Şekil 4.4

Çok Çekirdek Programı Otomatik Olarak Hızlandırmaz

Çok çekirdekten yararlanmak isteyen programcının beş temel zorluğu:

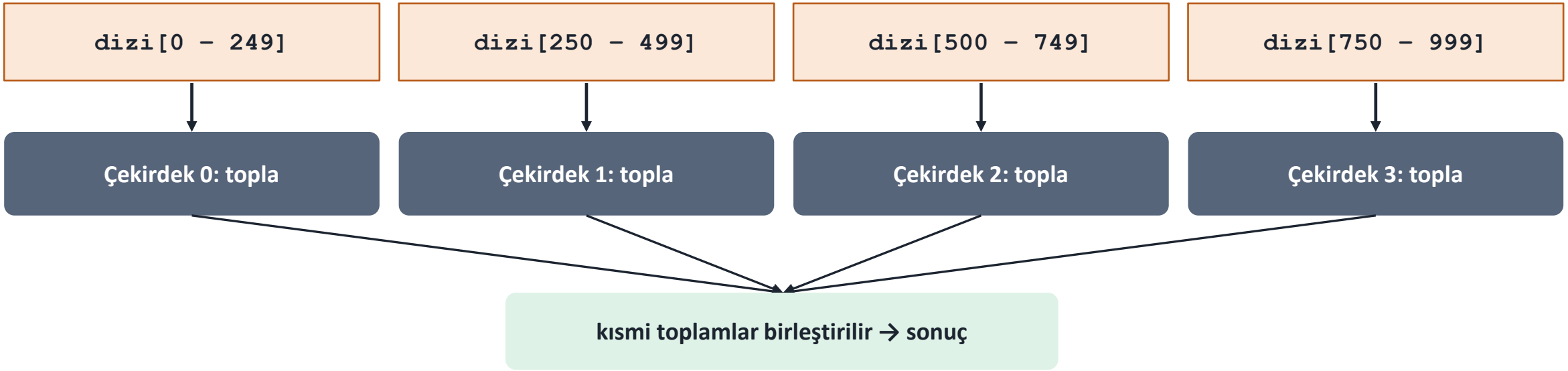
1	**Görevleri belirleme**	Programın eşzamanlı çalışabilecek, birbirinden bağımsız bölümlerini bulmak
2	**Denge**	Görevlerin benzer miktarda iş yapmasını sağlamak; az iş yapan görev için ayrı çekirdek israf olabilir
3	**Veriyi bölme**	Görevlerin eriştiği veriyi ayrı çekirdeklerde çalışacak biçimde bölmek
4	**Veri bağımlılığı**	Bir görev diğerinin sonucuna ihtiyaç duyuyorsa yürütme sırasını eşgüdümlemek (Hafta 6)
5	**Test ve hata ayıklama**	Paralel yürütmede çok sayıda olası yürütme yolu vardır; hatayı yeniden üretmek zordur

Tek iş parçacıklı bir program, 8 çekirdekli bir makinede de tek çekirdek kullanır.

Veri Paralelliği (Data Parallelism)

Aynı işlem, verinin farklı parçalarına farklı çekirdeklerde uygulanır.

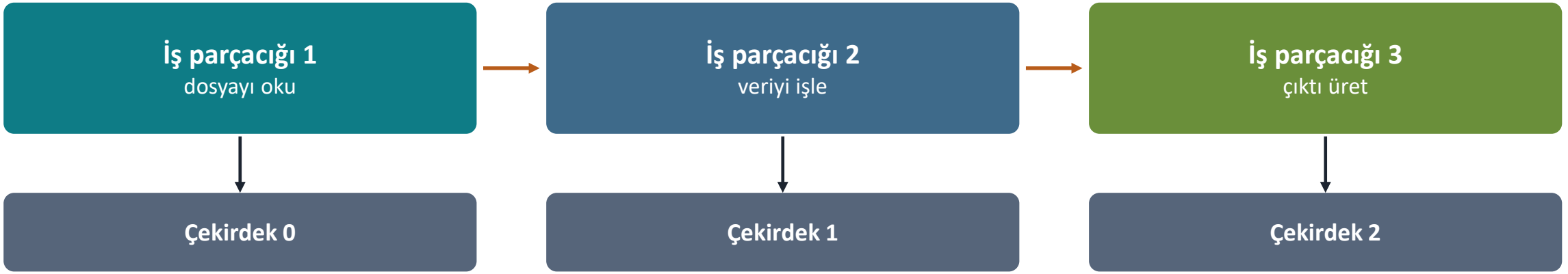
Örnek: 1000 elemanlı dizinin toplamı, 4 çekirdek



Kaynak: Silberschatz vd., Bölüm 4.2.2 (kitap N elemanlı dizi ve iki çekirdekle anlatır)

Görev Paralelliği (Task Parallelism)

Farklı görevler (iş parçacıkları) farklı çekirdeklere dağıtılır; her biri farklı bir işlem yapar.

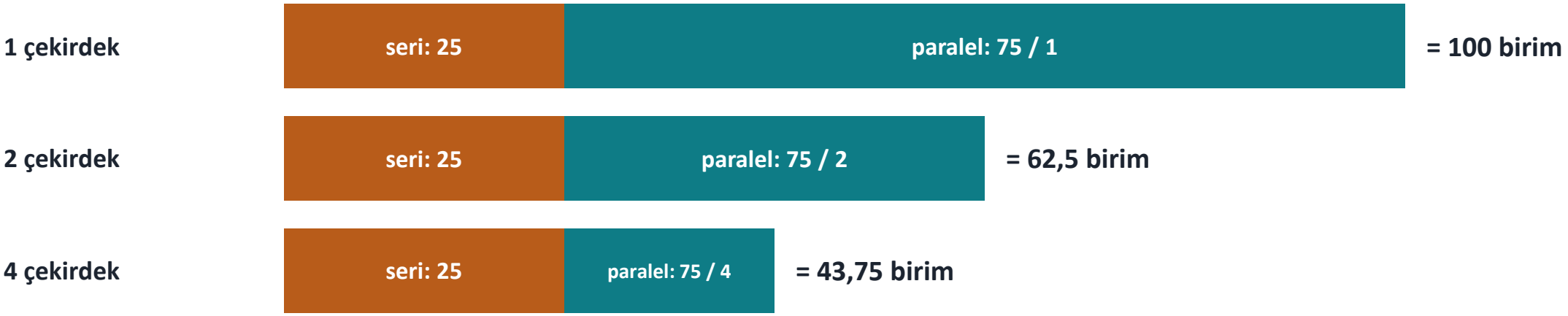


Ölçüt	Veri paralelliği	Görev paralelliği
Ne dağıtılır?	Veri	İşlemler (görevler)
Çekirdekler ne yapar?	Aynı işlemi	Farklı işlemleri

Turuncu oklar: veri bir aşamadan diğerine akar · Uygulamalar çoğunlukla iki yaklaşımın karışımını kullanır. Silberschatz vd., Bölüm 4.2.2

Amdahl Yasası: Sezgisel Bakış

Bir programın toplam süresi = seri bölüm + paralelleştirilebilir bölüm



Gözlem

Çekirdek eklendikçe yalnızca paralel bölüm kısalır. Seri bölüm (25 birim) hiç küçülmez.

Soru

Sonsuz çekirdek olsa bile süre 25 birimin altına inebilir mi? → Hızlanmanın bir üst sınırı vardır.

Örnek: %75 paralelleştirilebilir, %25 seri program (100 birimlik iş)

Amdahl Yasası: Formül

$$\text{Hızlanma}(N) \leq \frac{1}{(1 - P) + P / N}$$

P	programın paralelleştirilebilir oranı (0 ile 1 arası)
1 – P	seri (paralelleştirilemeyen) oran
N	işlem çekirdeği sayısı

Formül nereden geliyor?

Tek çekirdekte süre **1 (normalize)**	N çekirdekte seri bölüm **1 – P (değişmez)**	N çekirdekte paralel bölüm **P / N (N parçaya bölünür)**	Hızlanma = eski süre / yeni süre **1 / [(1 – P) + P/N]**

"≤" işareti: Bu, iş parçacığı oluşturma ve eşgüdüm ek yükü yok sayılarak elde edilen teorik üst sınırdır.

Kaynak: Silberschatz vd., Bölüm 4.2 (Amdahl Yasası)

Çözümlü Örnek 1: %75 Paralel, 4 Çekirdek

Verilenler: P = 0,75 · N = 4

1. **Seri oranı bul** — $1 - P = 1 - 0{,}75 = 0{,}25$
2. **Paralel bölümü N'ye böl** — $P / N = 0{,}75 / 4 = 0{,}1875$
3. **Paydayı topla** — $0{,}25 + 0{,}1875 = 0{,}4375$
4. **Tersini al** — $1 / 0{,}4375 \approx 2{,}29$

Sonuç

≈ 2,29 kat

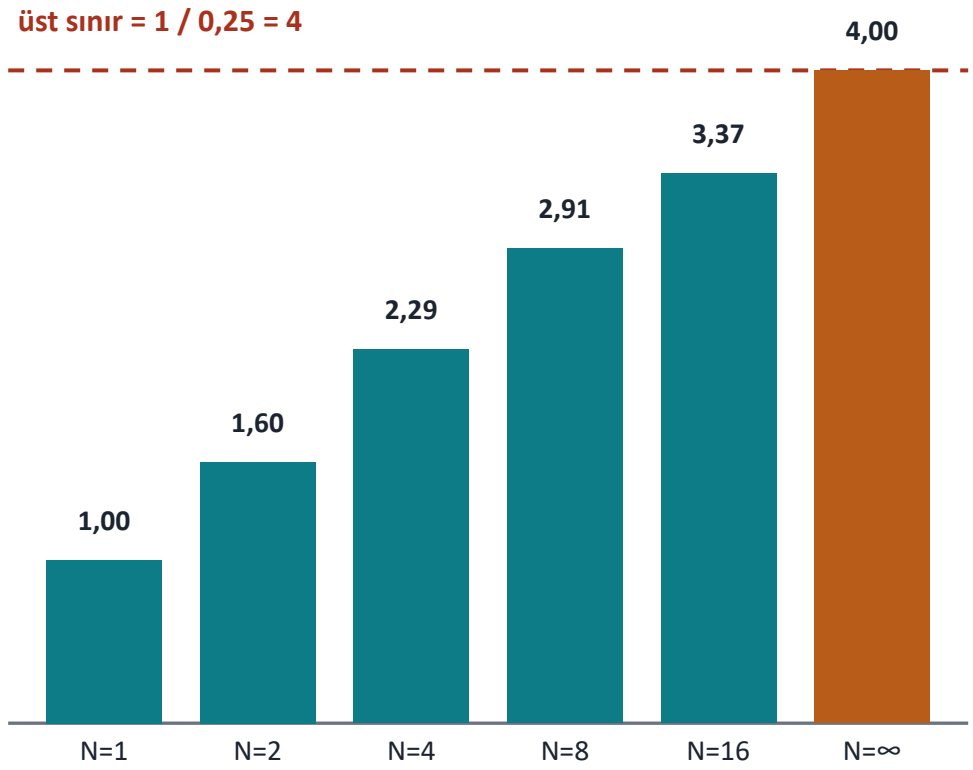
4 çekirdek, 4 kat değil yaklaşık 2,29 kat hızlanma sağlar.

Kontrol: aynı program 2 çekirdekte → 1 / (0,25 + 0,375) = 1 / 0,625 = 1,6 kat (kitaptaki örnek).

Kitap 4 çekirdek için 2,28 verir (kesilmiş değer); tam değer 2,2857…

Çözümlü Örnek 2: Çekirdek Sayısı Sonsuza Giderse

N	Payda: 0,25 + 0,75/N	Hızlanma
1	0,25 + 0,75 = 1	1,00
2	0,25 + 0,375 = 0,625	1,60
4	0,25 + 0,1875 = 0,4375	2,29
8	0,25 + 0,09375 = 0,34375	2,91
16	0,25 + 0,046875 = 0,296875	3,37
N → ∞	**0,25 + 0 = 0,25**	**4,00**



N → ∞ iken P/N → 0; hızlanma 1 / (1 − P) değerine yaklaşır. Seri bölüm, en büyük teorik hızlanmayı sınırlar.

Çözümlü Örnek 3: %90 Paralel, 8 Çekirdek

Verilenler: P = 0,90 · N = 8

Seri oran	$1 - 0{,}90 = 0{,}10$
Paralel / N	$0{,}90 / 8 = 0{,}1125$
Payda	$0{,}10 + 0{,}1125 = 0{,}2125$
Hızlanma	$1 / 0{,}2125 \approx 4{,}71$

Üst sınır: N → ∞

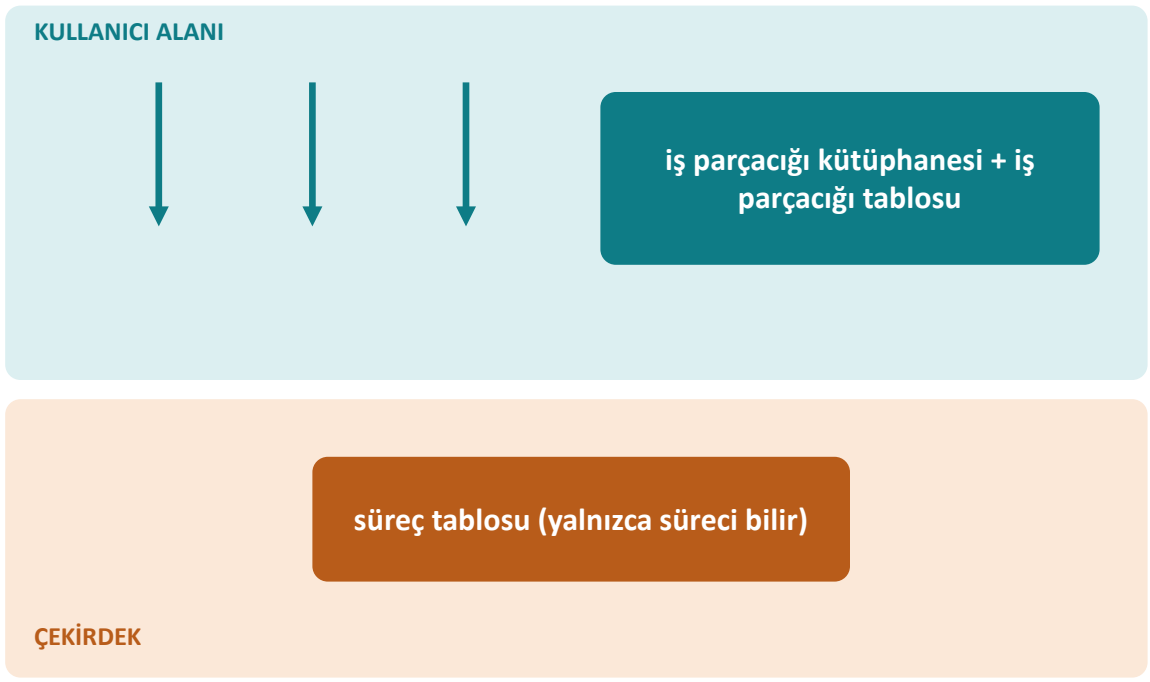
$$1 / (1 - 0{,}90) = 1 / 0{,}10 = 10$$

- Kodun %90'ı paralel olsa bile, sınırsız çekirdekle en fazla 10 kat hızlanma
- 8 çekirdek bu sınırın yarısına bile ulaşamaz (≈ 4,71)
- Seri bölümü küçültmek, çekirdek eklemekten daha etkili olabilir

Amdahl Yasası teorik bir üst sınırdır; gerçek hızlanma iş parçacığı yönetimi, eşgüdüm ve bellek erişimi gibi ek maliyetlerle daha düşük olabilir.

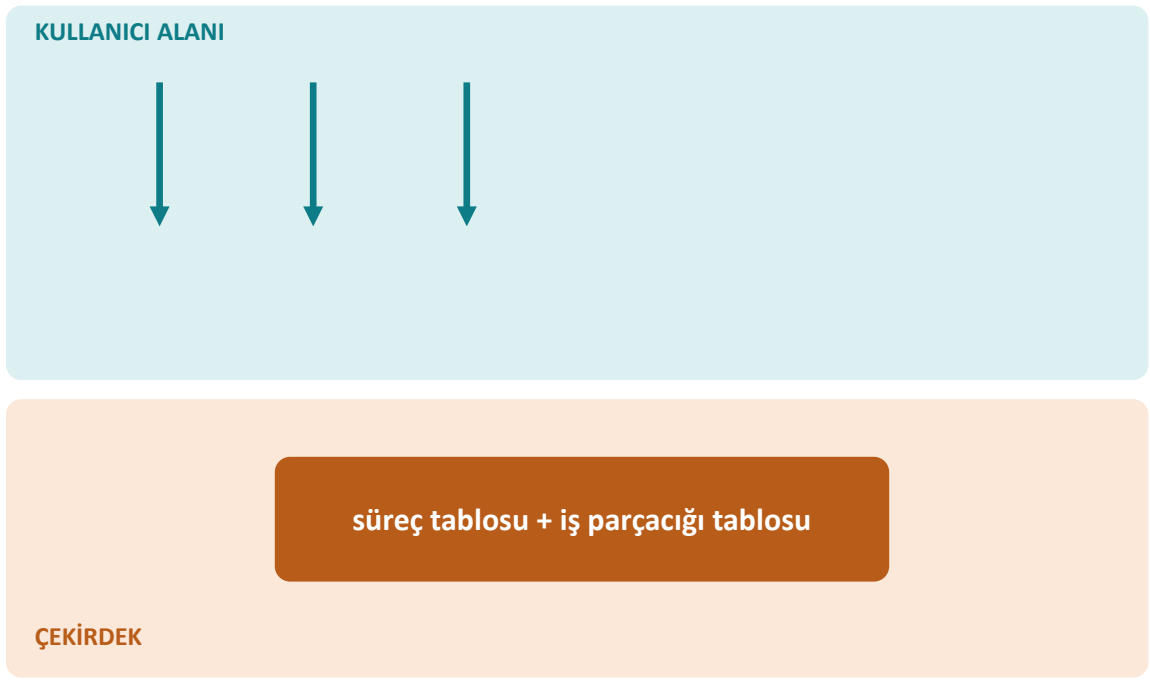
Kullanıcı Düzeyi ve Çekirdek Düzeyi İş Parçacıkları

Kullanıcı düzeyi (user-level)



- Kullanıcı alanındaki bir kütüphane yönetir
- Çekirdek her iş parçacığını ayrı ayrı bilmeyebilir

Çekirdek düzeyi (kernel-level)



- Çekirdek doğrudan destekler ve yönetir
- CPU Zamanlayıcısı onları ayrı yürütme birimleri olarak görür

Tanenbaum & Bos, Bölüm 2.2.4–2.2.5 (Şekil 2-16) temel alınmıştır

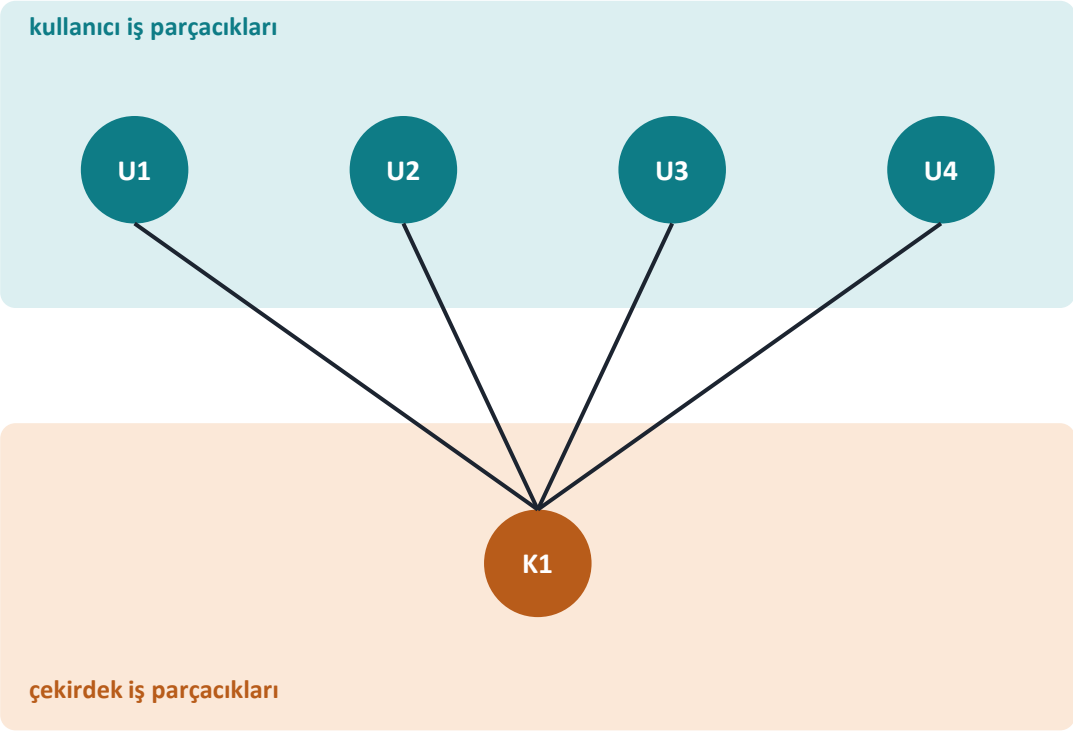
Kullanıcı ve Çekirdek İş Parçacıkları: Ödünleşimler

Ölçüt	Kullanıcı düzeyi	Çekirdek düzeyi
İş parçacığı geçişi	Çekirdeğe tuzak gerekmez; genellikle çok hızlı olabilir	Çekirdek üzerinden yapılır; maliyeti daha yüksek olabilir
Zamanlama	Uygulamaya özgü zamanlama algoritması kullanılabilir	Çekirdeğin zamanlama politikası uygulanır
Engelleyen sistem çağrısı	Bir iş parçacığı engellenirse tüm süreç engellenebilir	Yalnızca o iş parçacığı engellenir
Çok çekirdek	Çekirdek tek birim gördüğünden paralellik sınırlı olabilir	Farklı çekirdeklerde paralel çalışabilir
Çekirdek desteği	Desteklemeyen İS'de de gerçekleştirilebilir	Çekirdek desteği gerekir

Hangisinin daha iyi olduğu bağlama ve eşleme modeline bağlıdır. Sonuçta kullanıcı iş parçacıkları bir biçimde çekirdek iş parçacıklarına eşlenmelidir.

Kaynaklar: Tanenbaum & Bos, Bölüm 2.2.4–2.2.5; Silberschatz vd., Bölüm 4.3

Çoktan-Bire Modeli (Many-to-One)



Birçok kullanıcı iş parçacığı tek bir çekirdek iş parçacığına eşlenir.

Olası artı

İş parçacığı yönetimi kullanıcı alanında yapılır; maliyeti düşük olabilir.

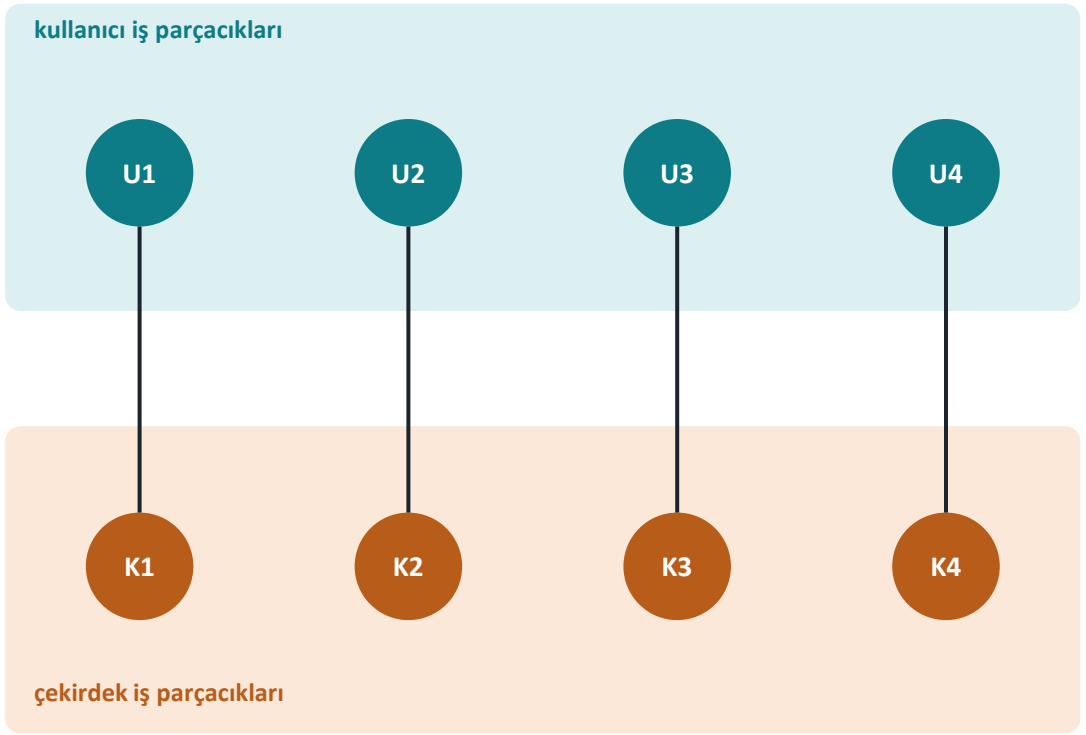
Temel sınırlamalar

- Bir iş parçacığı engelleyen sistem çağrısı yaparsa tüm süreç engellenir
- Tek çekirdek iş parçacığı olduğundan çok çekirdekte paralel çalışamaz

Örnekler (kitapta): Solaris green threads, GNU Portable Threads. Çok çekirdekten yararlanamadığı için bugün az kullanılır.

Silberschatz vd., Bölüm 4.3.1 ve Şekil 4.5

Birebir Modeli (One-to-One)



Her kullanıcı iş parçacığı ayrı bir çekirdek iş parçacığına eşlenir.

Artılar

- Bir iş parçacığı engellense de diğerleri çalışabilir
- Çok çekirdekte iş parçacıkları paralel çalışabilir

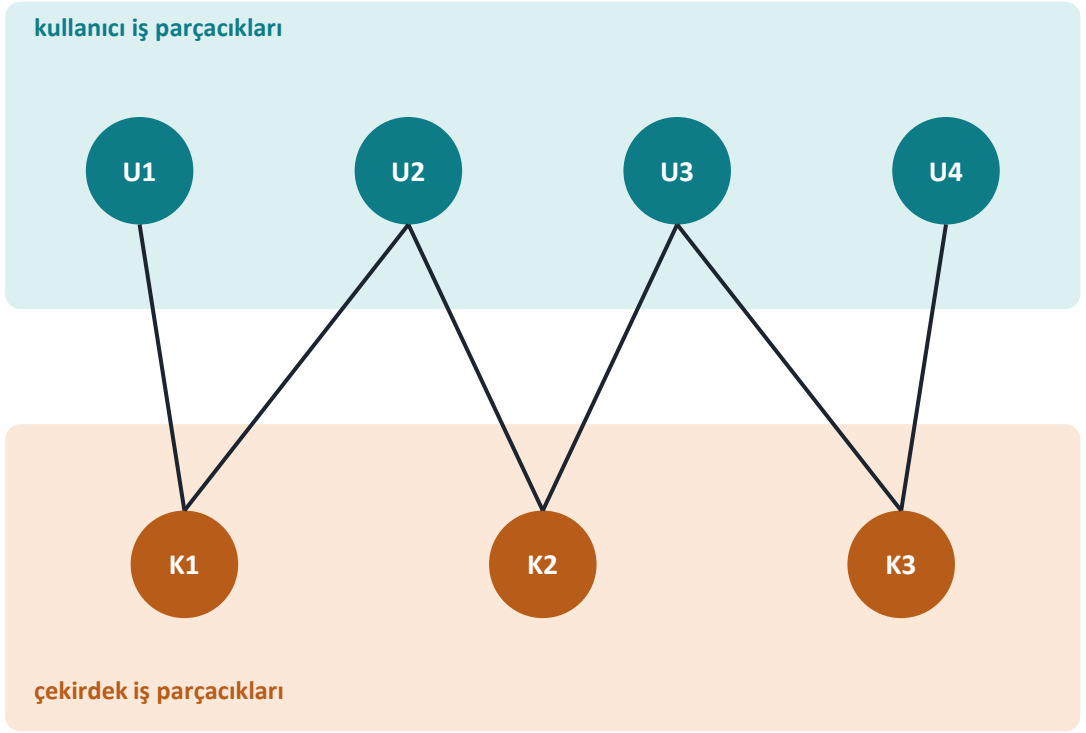
Bedel

Her kullanıcı iş parçacığı için çekirdek iş parçacığı gerekir; bu ek yük nedeniyle birçok sistem iş parçacığı sayısını sınırlar.

Örnekler (kitapta): Linux ve Windows işletim sistemleri ailesi birebir modelini gerçekleştirir.

Silberschatz vd., Bölüm 4.3.2 ve Şekil 4.6

Çoktan-Çoka Modeli (Many-to-Many)



Birçok kullanıcı iş parçacığı, daha az ya da eşit sayıda çekirdek iş parçacığı üzerinde çoğullanır.

Amaç: iki dünyanın iyi yanları

- Geliştirici istediği kadar kullanıcı iş parçacığı oluşturabilir
- Karşılık gelen çekirdek iş parçacıkları çok çekirdekte paralel çalışabilir
- Bir iş parçacığı engellenince çekirdek başka birini zamanlayabilir

Bedel: gerçekleştirmesi karmaşıktır.

Not: Çizgiler belirli bir andaki eşlemeyi gösterir; kullanıcı iş parçacıkları zaman içinde farklı çekirdek iş parçacıklarına eşlenebilir.

Silberschatz vd., Bölüm 4.3.3 ve Şekil 4.7

Modellerin Karşılaştırması ve İki Düzeyli Model

Ölçüt	Çoktan-bire	Birebir	Çoktan-çoka
Eşleme	N kullanıcı → 1 çekirdek	1 kullanıcı → 1 çekirdek	N kullanıcı → M çekirdek (M ≤ N)
Engelleyen çağrının etkisi	Tüm süreç engellenir	Yalnızca o iş parçacığı	Yalnızca o iş parçacığı
Çok çekirdekte paralellik	Yok	Var	Var
İş parçacığı sayısı	Kütüphaneye bağlı	Sistem sınırlayabilir	Esnek
Karmaşıklık	Düşük	Orta	Yüksek

İki düzeyli model (two-level): Çoktan-çoka modelin bir çeşidi; ayrıca bir kullanıcı iş parçacığının bir çekirdek iş parçacığına sabit olarak bağlanmasına (bound) izin verir.

Kaynak: Silberschatz vd., Bölüm 4.3 (Şekil 4.8: iki düzeyli model)

İş Parçacığı Kütüphanesi Nedir?

Programcıya iş parçacığı oluşturma ve yönetme için API sağlar.

Tamamen kullanıcı alanında

Kod ve veri yapıları kullanıcı alanındadır; bir kütüphane işlevini çağırmak yerel bir işlev çağrısıdır, sistem çağrısı değildir.

Çekirdek düzeyinde

İS'nin desteklediği kütüphane; API işlevini çağırmak genellikle çekirdeğe sistem çağrısıyla sonuçlanır.

POSIX Pthreads
Kullanıcı ya da çekirdek düzeyi olabilir · UNIX, Linux, macOS

Windows iş parçacıkları
Çekirdek düzeyi kütüphane · Windows

Java iş parçacıkları
JVM içinde; genellikle ana makinedeki İS kütüphanesiyle gerçekleştirilir

Eşzamansız iş parçacığı: ebeveyn çocuk iş parçacığını oluşturup kendi işine devam eder · Eşzamanlı: ebeveyn çocuklarının bitmesini bekler (join).

Kaynak: Silberschatz vd., Bölüm 4.4

Pthreads: Temel Kavramlar ve İlk Örnek

```c
#include <pthread.h>
#include <stdio.h>

void *merhaba(void *arg) {
    printf("İş parçacığından merhaba\n");
    return NULL;
}

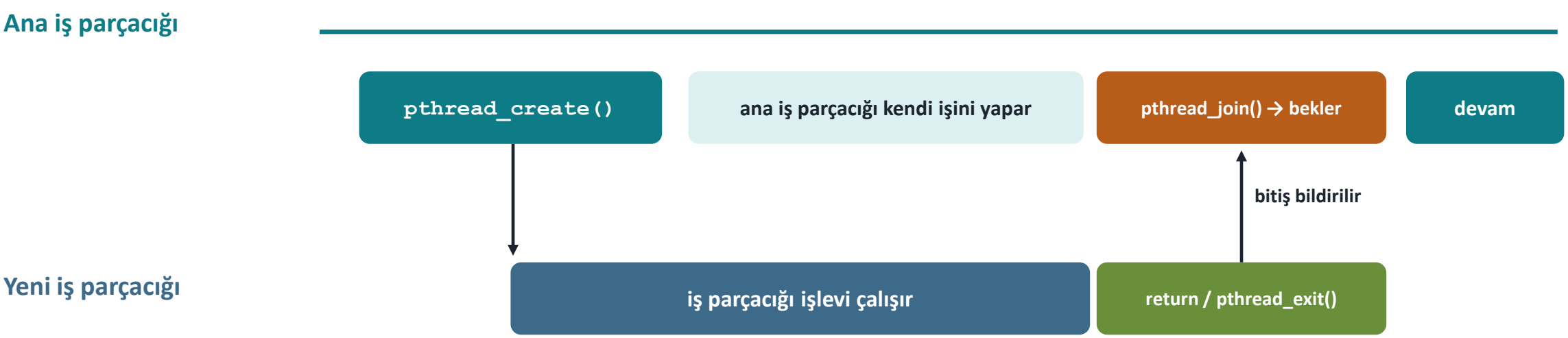
int main(void) {
    pthread_t tid;
    pthread_create(&tid, NULL, merhaba, NULL);
    pthread_join(tid, NULL);
    printf("Ana iş parçacığı bitti\n");
    return 0;
}
```

`pthread_t`	iş parçacığı kimliği türü
iş parçacığı işlevi	void* alır, void* döndürür; yürütme burada başlar
`pthread_create()`	yeni iş parçacığı oluşturur
`pthread_join()`	belirtilen iş parçacığının bitmesini bekler

```
$ gcc ornek.c -pthread
```

Silberschatz vd., Bölüm 4.4.1 temel alınarak sadeleştirilmiştir

Pthreads Yaşam Döngüsü: Oluştur, Çalıştır, Bekle (join)



join olmazsa?

main döndüğünde süreç sonlanır; henüz bitmemiş iş parçacıkları da sonlanır. Ana iş parçacığı sonucu kullanacaksa join ile beklemelidir.

Hafta 3 ile benzerlik

pthread_create / pthread_join ilişkisi, süreçlerdeki fork / wait ilişkisine benzer — ama yeni adres uzayı oluşmaz.

Kavramsal zaman çizelgesi · Silberschatz vd., Bölüm 4.4.1

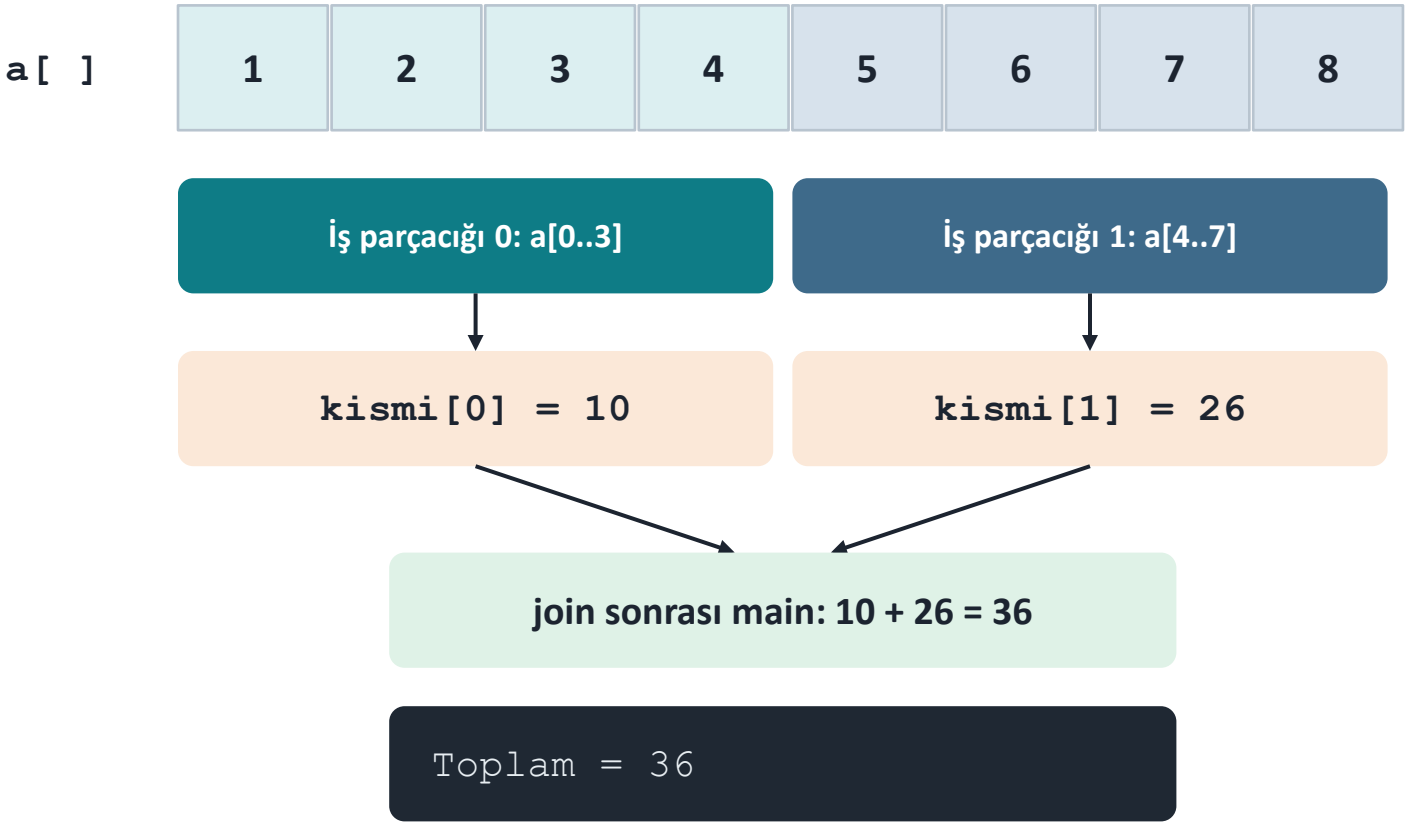
Çözümlü Uygulama: İki İş Parçacığıyla Dizi Toplamı (Kod)

```
#include <pthread.h>
#include <stdio.h>
int a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
int kismi[2];               /* her iş parçacığına bir hücre */

void *topla(void *arg) {
    int id = *(int *)arg, t = 0;
    for (int i = id * 4; i < id * 4 + 4; i++) t += a[i];
    kismi[id] = t;
    return NULL;
}
int main(void) {
    pthread_t th[2];  int id[2] = {0, 1};
    for (int i = 0; i < 2; i++) pthread_create(&th[i], NULL, topla, &id[i]);
    for (int i = 0; i < 2; i++) pthread_join(th[i], NULL);
    printf("Toplam = %d\n", kismi[0] + kismi[1]);
    return 0;
}
```

Yürütme analizi bir sonraki slaytta

Çözümlü Uygulama: Yürütme Analizi



- Veri paralelliği: aynı işlem, dizinin iki yarısına uygulanır
- Her iş parçacığı yalnızca kendi kismi[id] hücresine yazar: aynı konuma yazan iki iş parçacığı yoktur
- id dizisi main'de durur; her iş parçacığına kendi elemanının adresi verilir
- main, iki join'den sonra toplar; böylece kısmi sonuçlar hazırdır

Aynı değişkeni birden çok iş parçacığının güncellemesi farklı bir sorundur → Hafta 6 (yarış durumu)

Windows ve Java İş Parçacıkları: Kısa Karşılaştırma

Windows API (C)

```
DWORD WINAPI Gorev(LPVOID p) {
    /* iş */  return 0;
}
HANDLE h = CreateThread(NULL, 0, Gorev,
                        &veri, 0, &tid);
WaitForSingleObject(h, INFINITE);
CloseHandle(h);
```

Java

```
class Gorev implements Runnable {
    public void run() { /* iş */ }
}
Thread t = new Thread(new Gorev());
t.start();   // run() yeni iş parçacığında
t.join();    // bitmesini bekle
```

Kavram	Pthreads	Windows	Java
Oluştur	pthread_create()	CreateThread()	new Thread(...).start()
Bekle	pthread_join()	WaitForSingleObject()	join()

Kaynak: Silberschatz vd., Bölüm 4.4.2–4.4.3 · Java'da join() InterruptedException yakalanmalıdır

Yüzlerce İş Parçacığını Programcı mı Yönetmeli?

- Uygulamalar yüzlerce, hatta binlerce iş parçacığı içerebilir
- Her birini elle oluşturmak, beklemek ve doğruluğunu sağlamak zorlaşır

Örtük iş parçacığı: İş parçacıklarının oluşturulması ve yönetimi uygulama geliştiricisinden alınıp derleyicilere ve çalışma zamanı kütüphanelerine devredilir.

İş parçacığı havuzu
Thread Pool

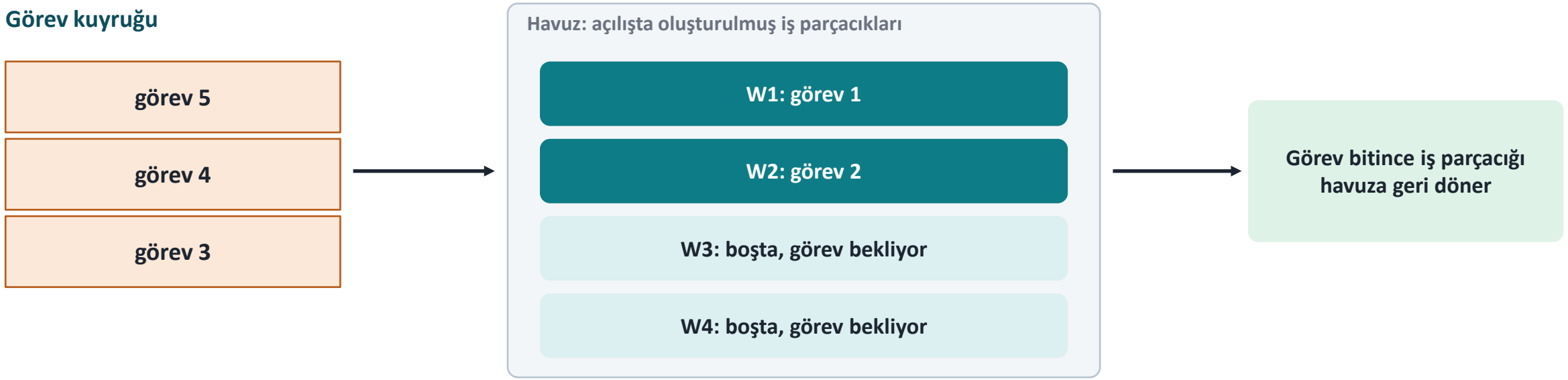
OpenMP
derleyici yönergeleri + API

Grand Central Dispatch
macOS / iOS (kısaca)

Programcı paralel çalışabilecek görevleri belirler; iş parçacıklarına nasıl eşleneceğine kütüphane ya da derleyici karar verir.

Kaynak: Silberschatz vd., Bölüm 4.5

İş Parçacığı Havuzu (Thread Pool)



Faydalar

Hız: Var olan iş parçacığıyla hizmet vermek, yenisini oluşturmaktan genellikle hızlıdır

Sınır: Aynı anda var olan iş parçacığı sayısı sınırlanır; sistem kaynakları korunur

Ayrım: Görev ile onu yürütme mekanizması ayrılır; farklı yürütme stratejileri mümkün olur

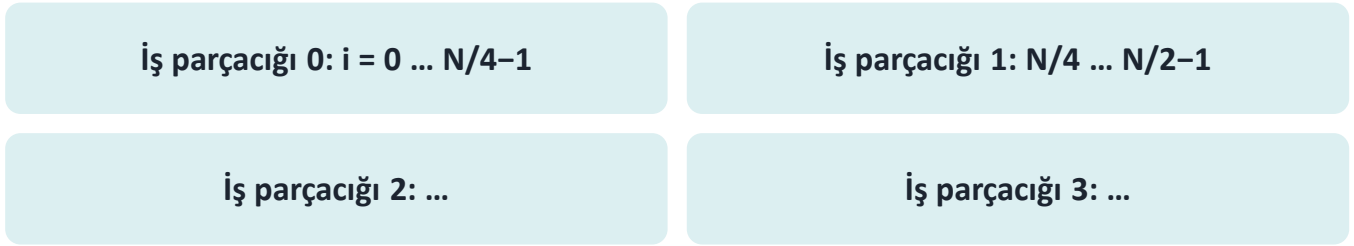
Havuz her görev için yeni iş parçacığı oluşturmaz. Kaynak: Silberschatz vd., Bölüm 4.5.1

OpenMP: Derleyici İş Parçacıklarını Üstlenir

```
#include <omp.h>

/* döngü iş parçacıklarına bölünür */
#pragma omp parallel for
for (i = 0; i < N; i++)
    c[i] = a[i] + b[i];
```

- Programcı paralel bölgeleri derleyici yönergeleriyle (#pragma) işaretler
- OpenMP çalışma zamanı, sistemdeki çekirdek sayısı kadar iş parçacığı oluşturur
- Döngü yinelemeleri iş parçacıkları arasında bölünür: veri paralelliği
- İş parçacığı oluşturma, bölme ve bekleme kodu programcıya görünmez



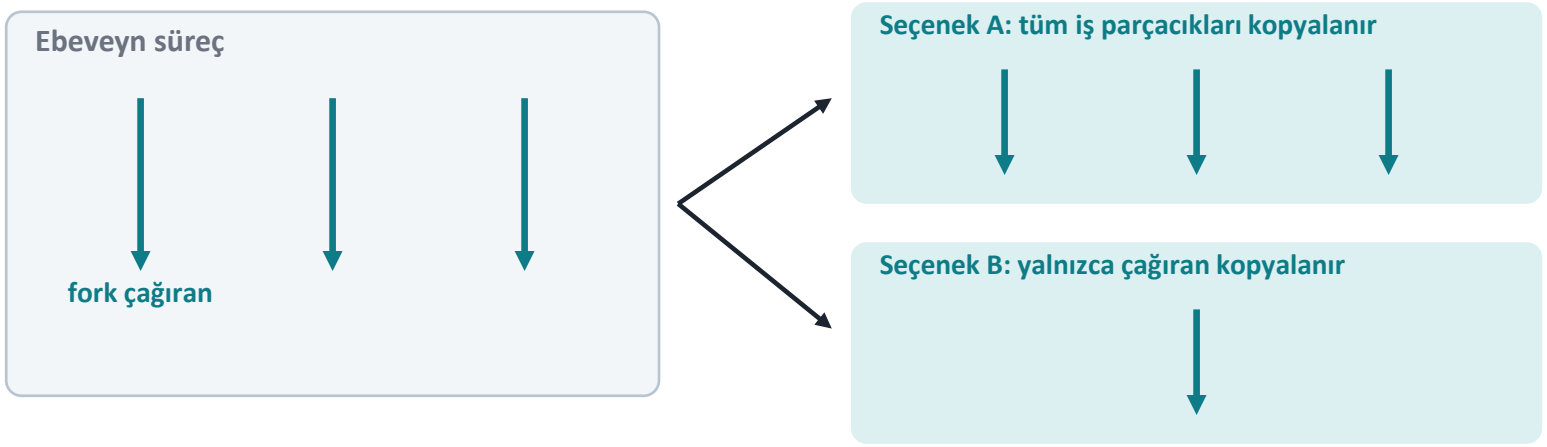
Örnek 4 çekirdek için kavramsal bölüştürmedir; gerçek bölüştürme çalışma zamanına bağlıdır. Silberschatz vd., Bölüm 4.5.2

Çok İş Parçacıklı Programlarda Yeni Sorular

1	**fork() ve exec()**	Çok iş parçacıklı süreç fork() çağırırsa çocuk kaç iş parçacığı içerir?
2	**Sinyal işleme**	Sürece gelen bir sinyal hangi iş parçacığına iletilmeli?
3	**İş parçacığı iptali**	Bir iş parçacığını işi bitmeden güvenle nasıl sonlandırırız?
4	**İş parçacığına özel depolama**	Her iş parçacığının aynı adlı ama kendine ait verisi nasıl tutulur?
5	**Zamanlayıcı etkinleştirmeleri**	Kullanıcı kütüphanesi ile çekirdek nasıl eşgüdüm kurar?

Kaynak: Silberschatz vd., Bölüm 4.6 · Bu bölüm genel bakış düzeyindedir

Çok İş Parçacıklı Süreçte fork() ve exec()



- Kitap: bazı UNIX sistemleri iki farklı fork() sürümü sunar
- POSIX fork(): çocukta yalnızca çağıran iş parçacığı bulunur
- Hangi sürümün uygun olduğu uygulamaya bağlıdır

exec() ne yapar?

Hafta 3'teki gibi: süreç görüntüsünün tamamını değiştirir — dolayısıyla tüm iş parçacıkları ortadan kalkar ve yeni program tek iş parçacığıyla başlar.

Pratik sonuç

fork()'tan hemen sonra exec() çağrılacaksa tüm iş parçacıklarını kopyalamak gereksizdir; yalnızca çağıran iş parçacığını kopyalamak yeterlidir.

Kaynak: Silberschatz vd., Bölüm 4.6.1

Sinyal İşleme (Signal Handling)

Sinyal: UNIX'te bir sürece belirli bir olayın gerçekleştiğini bildiren mekanizma. Üretilir → sürece iletilir → işlenir.

Eşzamanlı sinyal

Sinyale neden olan işlemi yapan iş parçacığına iletilir (ör. geçersiz bellek erişimi, sıfıra bölme).

Eşzamansız sinyal

Süreç dışındaki bir olaydan gelir (ör. Ctrl+C ile sonlandırma); hangi iş parçacığına iletileceği açık değildir.

Çok iş parçacıklı süreçte olası iletim seçenekleri

- **Sinyalin ilgili olduğu iş parçacığına**
- **Süreçteki her iş parçacığına**
- **Belirli iş parçacıklarına**
- **Tüm sinyalleri alan belirli bir iş parçacığına**

Kaynak: Silberschatz vd., Bölüm 4.6.2 · POSIX'te belirli iş parçacığına sinyal: pthread_kill()

İş Parçacığı İptali (Thread Cancellation)

Hedef iş parçacığını (target thread) işi bitmeden sonlandırmak. Örnek: birden çok iş parçacığı veritabanında arama yapar; biri sonucu bulunca diğerleri iptal edilebilir.

Eşzamansız iptal (asynchronous)

- Hedef iş parçacığı hemen sonlandırılır
- Sorun: paylaşılan veriyi güncellemenin ortasında olabilir; ayrılmış kaynaklar geri alınmayabilir

Ertelenmiş iptal (deferred)

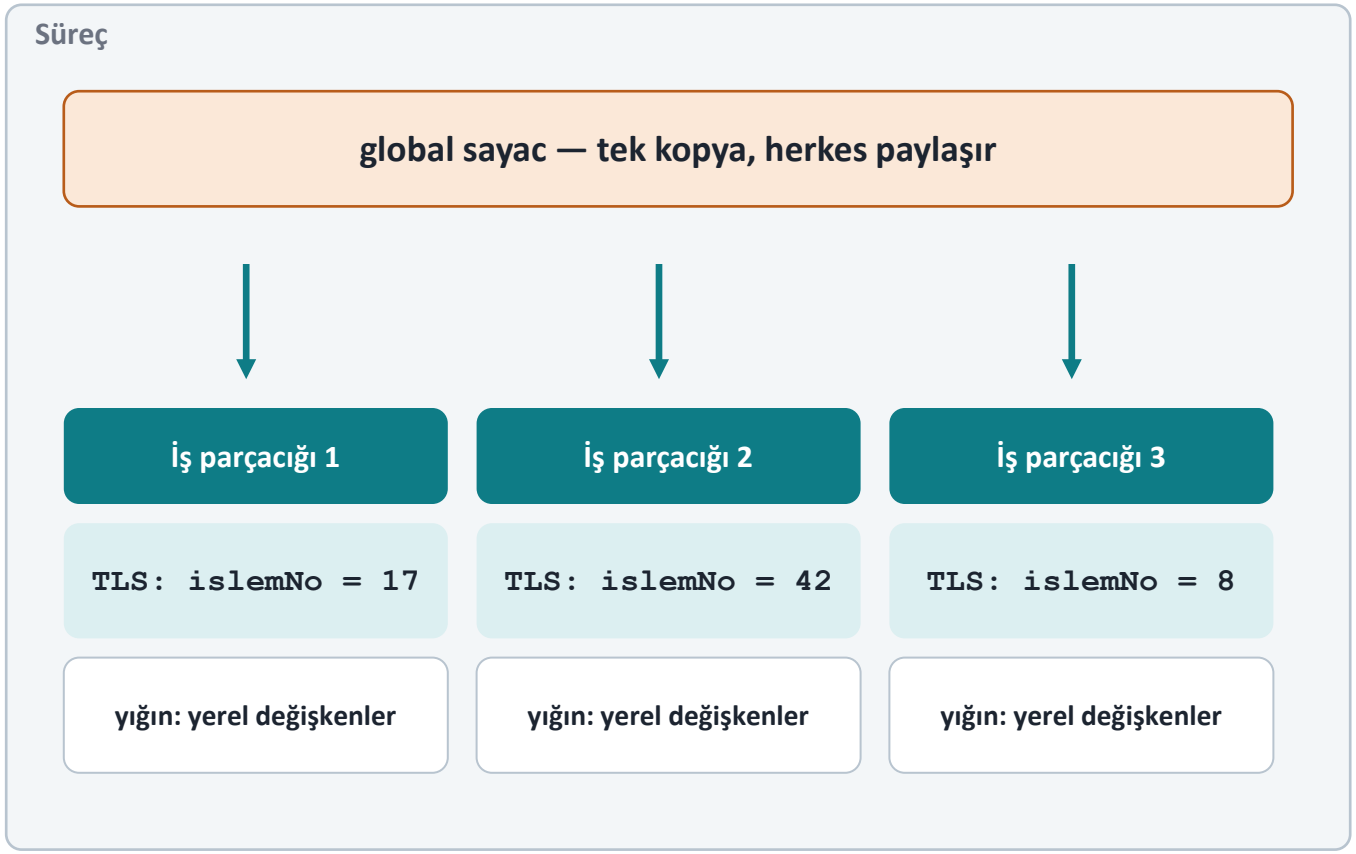
- Hedef iş parçacığı sonlanması gerekip gerekmediğini düzenli olarak denetler
- İptal yalnızca iptal noktasına (cancellation point) ulaşınca gerçekleşir → güvenli bir noktada kendini sonlandırır

```
pthread_cancel(tid);     /* iptal isteği gönder      */
pthread_testcancel();    /* hedefte: iptal noktası   */
```

Pthreads'te varsayılan tür ertelenmiş iptaldir.

Kaynak: Silberschatz vd., Bölüm 4.6.3

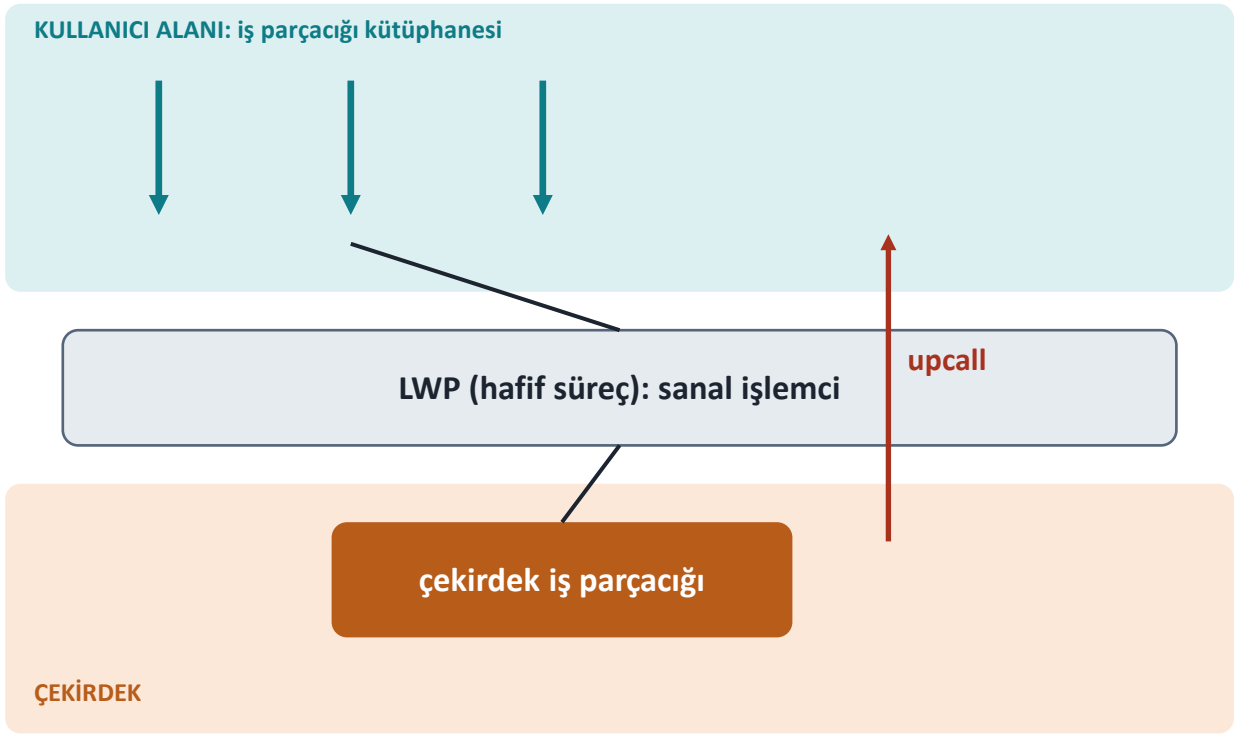
İş Parçacığına Özel Depolama (Thread-Local Storage, TLS)



- Her iş parçacığı aynı adlı verinin kendi kopyasına sahiptir
- Yerel değişkenden farkı: tek bir işlev çağrısıyla sınırlı değildir, işlevler arasında görünür
- Global değişkenden farkı: iş parçacıkları arasında paylaşılmaz
- Örnek: her işlem (transaction) için benzersiz bir kimliği tutmak

Kaynak: Silberschatz vd., Bölüm 4.6.4 · Pthreads, Windows ve Java TLS desteği sağlar

Zamanlayıcı Etkinleştirmeleri (Scheduler Activations)



Sorun: Kullanıcı kütüphanesi çekirdeğin ne yaptığını bilmezse uygulama en iyi başarımı alamayabilir.

- Çoktan-çoka ve iki düzeyli modellerde kullanıcı iş parçacıkları LWP adlı ara yapılar üzerinden çekirdek iş parçacıklarına bağlanır
- Çekirdek uygulamaya sanal işlemciler (LWP'ler) sağlar
- Önemli olaylarda (ör. bir iş parçacığı engellenecek) çekirdek kütüphaneye upcall ile haber verir; kütüphane başka bir iş parçacığını çalıştırabilir

Kaynaklar: Silberschatz vd., Bölüm 4.6.5 (Şekil 4.13); Tanenbaum & Bos, Bölüm 2.2.7 · Zamanlama algoritmaları Hafta 5'te

Gerçek Sistem Örneği: Windows İş Parçacıkları

Süreç = kaynak kabı (resource container) · İş parçacığı = çekirdeğin CPU'yu zamanladığı birim

Her iş parçacığının bileşenleri

- **İş parçacığı kimliği**
- **Yazmaç kümesi (işlemci durumu)**
- **Kullanıcı yığını + çekirdek yığını**
- **Özel depolama alanı (kütüphaneler, DLL'ler için)**

Yazmaçlar, yığınlar ve özel depolama = iş parçacığının bağlamı

Temel veri yapıları

ETHREAD — yürütme iş parçacığı bloğu: ait olduğu süreç, başlangıç adresi
çekirdek alanı

KTHREAD — çekirdek iş parçacığı bloğu: zamanlama ve eşgüdüm bilgisi, çekirdek yığını
çekirdek alanı

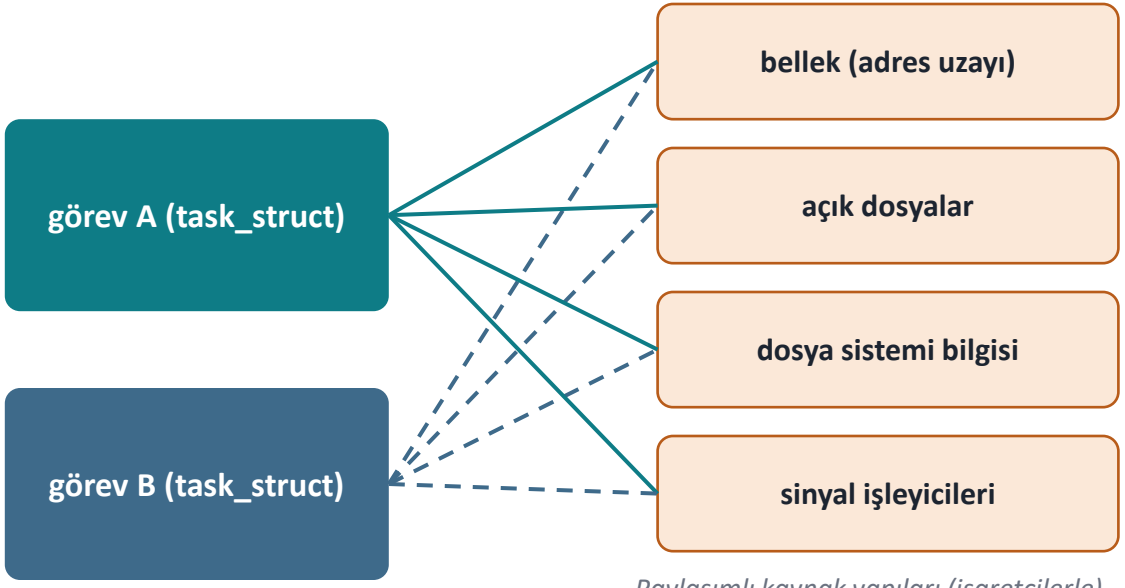
TEB — iş parçacığı ortam bloğu: kimlik, kullanıcı yığını, TLS
kullanıcı alanı

Windows birebir modeli kullanır: her kullanıcı iş parçacığı bir çekirdek iş parçacığına eşlenir.

Kitap temelli örnek: Silberschatz vd., Bölüm 4.7.1 (Windows 7); Tanenbaum & Bos, Bölüm 11.4.1

Gerçek Sistem Örneği: Linux'ta Görev (Task) ve clone()

- Linux süreç ile iş parçacığını ayırmaz; ikisine de görev (task) der
- fork(): ebeveynin kopyası olan yeni görev
- clone(): yeni görev oluştururken ebeveynle hangi kaynakların paylaşılacağı bayraklarla seçilir
- Hiç paylaşım yoksa fork() gibi; adres uzayı, dosyalar ve sinyal işleyicileri paylaşılırsa iş parçacığı gibi davranır



Paylaşımlı kaynak yapıları (işaretçilerle)

İş parçacığı gibi clone(): görevler aynı kaynak yapılarına işaret eder · fork(): kaynaklar kopyalanır

Kitap temelli kavramsal şema; bayrak listesi verilmemiştir. Silberschatz vd., Bölüm 4.7.2; Tanenbaum & Bos, Bölüm 10.3.3

Bu Hafta Neler Öğrendik?

1. İş parçacığı: süreç içinde ayrı PC, yazmaç ve yığına sahip yürütme akışı
2. İş parçacıkları kodu, veriyi, öbeği ve dosyaları paylaşır
3. Eşzamanlılık ≠ paralellik; paralellik çok çekirdek ister
4. Amdahl: seri bölüm hızlanmayı 1/(1−P) ile sınırlar
5. Çoktan-bire, birebir, çoktan-çoka eşleme modelleri
6. Pthreads, havuz, OpenMP; fork/exec, sinyal, iptal, TLS

Gelecek hafta: Hazır kuyruğunda bekleyen süreç ve iş parçacıklarından hangisi CPU'yu alacak? → CPU zamanlama

Yaygın Yanlış Anlamalar

YANLIŞ	DOĞRU
“Süreç = iş parçacığı”	Süreç kaynakların sahibidir; iş parçacığı süreç içindeki yürütme akışıdır
“Her iş parçacığının ayrı öbeği vardır”	Öbek süreç düzeyindedir ve paylaşılır; ayrı olan yığındır
“Eşzamanlılık = paralellik”	Tek çekirdekte eşzamanlılık olur ama paralellik olmaz
“Çok çekirdek varsa program otomatik hızlanır”	Program paralel görevlere bölünmelidir; seri bölüm sınır koyar
“İş parçacıkları tüm kaynakları ayrı tutar”	Kod, veri, öbek ve dosyalar paylaşılır
“Kullanıcı iş parçacığı her zaman daha hızlıdır”	Geçişi ucuz olabilir; ama engelleme ve paralellik sınırları vardır
“Çekirdek sayısı sonsuzsa hızlanma da sonsuzdur”	Hızlanma 1/(1−P) değerini aşamaz
“İş parçacığı havuzu her görev için yeni iş parçacığı oluşturur”	Önceden oluşturulmuş iş parçacıklarını yeniden kullanır

Kontrol Soruları

KAVRAMSAL · KARŞILAŞTIRMA

S1

Aynı süreç içindeki iki iş parçacığı hangi kaynakları paylaşır, hangilerini paylaşmaz?

İlgili bölüm: 1 · İş parçacığı kavramı

S2

Tek çekirdekli bir sistemde eşzamanlılık mümkünken gerçek paralellik neden mümkün değildir?

İlgili bölüm: 2 · Eşzamanlılık ve paralellik

DEĞERLENDİRME

Kontrol Soruları

AMDAHL HESABI

S3 **Bir programın %80'i paralelleştirilebilir. 4 çekirdek kullanılırsa teorik hızlanma nedir?**

Formül: 1 / [(1 − P) + P/N]

S4 **Aynı program için çekirdek sayısı sınırsız artırılırsa ulaşılabilecek en büyük hızlanma nedir? Neden?**

İpucu: N → ∞ iken P/N ne olur?

Kontrol Soruları

MODEL ANALİZİ

S5 **Çoktan-bire modelinin çok çekirdekli bir sistemdeki temel sınırlaması nedir?**

İlgili bölüm: 3 · Çoklu iş parçacığı modelleri

S6 **Birebir modelde bir iş parçacığı diskten okuma için engellenirse aynı süreçteki diğer iş parçacıklarına ne olur? Çoktan-bire modelde ne olurdu?**

İpucu: Çekirdek kaç ayrı yürütme birimi görüyor?

Kontrol Soruları

KISA KOD ANALİZİ

S7 **pthread_create() ile oluşturulan iş parçacığının bitmesini ana iş parçacığı hangi çağrıyla bekleyebilir? Bu çağrı yapılmazsa ne olabilir?**

İlgili bölüm: 4 · Pthreads

S8 **Dizi toplamı örneğinde iki iş parçacığı sonucu tek bir global toplam değişkenine eklese ne tür bir risk ortaya çıkabilirdi?**

Yalnızca riski adlandırın; çözümü Hafta 6'da göreceğiz.

Kaynaklar

ANA KAYNAK

Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating System Concepts* (9th ed.). John Wiley & Sons.

Bu hafta: Bölüm 4 — 4.1–4.7 (gerçek sistemler: 4.7.1 Windows, 4.7.2 Linux)

DESTEKLEYİCİ KAYNAK

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.

Bu hafta: Bölüm 2 — 2.2.1–2.2.7 · Gerçek sistemler: 10.3.3 (Linux), 11.4.1 (Windows)

Not: Bu sunumdaki diyagramlar, belirtilen yerlerde kaynak kitaplardaki şekiller temel alınarak yeniden çizilmiştir.